

Plan Your DSA Preparation in Eight Weeks

Study data structures through the operations they support and algorithms through the invariants that make them correct. Use topic-focused practice to learn, then mixed unseen problems to test recognition. A complete preparation loop includes clarification, implementation, testing, complexity, and revision.

For: Candidates who can implement simple programs and want a structured progression from foundational patterns to mixed interview problems, with enough review to retain and transfer the learning.

Suggested pace: 8 weeks, 6-10 hours per week | Basic programming required

What you will be able to do

- Choose an approach from constraints rather than a visible topic label.
- Explain the invariant and complexity of common interview patterns.
- Implement and test solutions without relying on memorized templates.
- Use a spaced error log and mixed mocks to measure transfer.

Build the right mental model

Constraints select techniques

Input size, ordering, value range, mutation rules, and output requirements change the solution. A quadratic method may be acceptable for small input; a sliding window may fail when negative values break monotonicity. Read the contract before naming a pattern.

Data structures encode operations

Maps support keyed lookup, heaps expose an extreme, queues process arrival order, and ordered structures support range reasoning. Compare the operations required by the problem with the actual cost and memory of the chosen structure.

Proof and testing complement each other

An invariant explains all valid executions; tests expose mistakes in the implementation or assumptions. Do both. A solution that passes a few examples can still be wrong, while a correct idea can be implemented with an off-by-one defect.

Study with evidence.

Use three focused problem sessions, one review session, and one mixed mock each week. Re-solve failures from a blank editor after a delay. Record the missed clue and invariant, not a copied solution paragraph or only a completion checkbox.

Weeks 1-2: Linear structures and search

Cover arrays, maps, sets, pointers, windows, stacks, and binary search. Trace indexes and counts explicitly. Practice both a simple baseline and one optimization, explaining what repeated work was removed.

Deliverable and check: Independent solutions with tests for minimal input, duplicates, boundaries, and invalid assumptions.

Weeks 3-4: Trees, heaps, and graphs

Study recursion, iterative traversal, BFS, DFS, priority queues, connected components, and topological order. Distinguish node identity from value and directed cycles from undirected parent edges.

Deliverable and check: A traversal notebook with visited timing, queue/stack state, and time complexity in vertices and edges.

Weeks 5-6: State and choice

Practice backtracking, greedy arguments, intervals, and dynamic programming. Define states before recurrences and justify greedy choices with an exchange or dominance argument. Compare memoization with bottom-up evaluation.

Deliverable and check: Several solved variants that demonstrate why a method applies, plus a counterexample where it does not.

Weeks 7-8: Mixed execution

Use unseen mixed sets and realistic time limits. Preserve time for tests and complexity. Add role-relevant harder problems only after medium-level correctness is stable. Review recurring failures and reduce new material near the interview.

Deliverable and check: A recent mock scorecard and a short revision list tied to observed weaknesses rather than a target problem count.

A repeatable practice session

Spend about 15 minutes recalling the previous topic without notes, 45 minutes on today's problem or lab, and 15 minutes explaining and checking the result. Record the failed assumption, not just the final answer. Revisit weak material after a few days and again the following week.

Apply it. Explain it.

Choose the pattern from a changed constraint

You are asked for a shortest contiguous subarray with sum at least a target. In one version every value is positive; in another values may be negative. The word 'subarray' alone does not determine the algorithm.

A worked approach

1. For positive values, adding to the right cannot decrease the sum and removing from the left cannot increase it. This supports expanding until valid and shrinking while preserving validity to minimize length.
2. Introduce a negative value and construct a case where removing an element increases the sum or extending lowers it. Explain exactly which monotone argument fails; do not reuse the same window mechanically.
3. Study an appropriate prefix-sum and monotonic-deque method for the general version, or state a correct slower baseline first. Compare contracts, complexity, and edge cases before implementing the advanced variant.

Expected result

The learning artifact is the distinction between the two contracts and their proofs. Recognizing why an approach stops applying is as valuable as remembering when it works.

Verify the result

- Can you state the monotonic property in the positive-input version?
- Does your counterexample actually break the reused window rule?
- Are empty results and impossible targets handled according to the problem?

Extend the exercise

Change 'shortest' to 'number of exact-sum subarrays.' Explain why prefix-frequency counts become useful and why storing only one prior prefix index would lose valid answers.

Make the learning stick.

Should I finish one topic completely before moving on?

Build enough understanding to implement and explain the basic pattern, then revisit it in mixed practice. There is no finite point where a topic is completely finished; spaced transfer is more useful than endless near-identical exercises.

How should I use solution explanations?

Attempt the problem, identify the exact blocker, read only enough to learn the missing idea, then implement independently and explain it. Revisit after a delay and test a variation to avoid mistaking recognition for mastery.

When should I practice hard problems?

When they teach a relevant new technique or stretch stable fundamentals. If basic implementation and testing remain unreliable, hard-problem memorization can hide rather than fix the main preparation gap.

Avoid these traps

- Selecting a pattern from the title while ignoring constraints.
- Counting copied solutions as independent practice.
- Skipping correctness reasoning and tests to maximize volume.

Ready for the next step?

- Mixed unseen problems can be approached without category hints.
- Each solution includes a clear invariant and complexity.
- Boundary cases are tested before completion is declared.
- The error log drives specific re-solves and variations.

Reliable next reads

- [MIT Introduction to Algorithms](#)
- [NeetCode pattern roadmap](#)
- [Python data structures](#)

An independent preparation guide, not an employer-endorsed syllabus or a guarantee of an offer. Company examples are practice contexts, not verified past interview questions. Use only public, synthetic, or authorized material in exercises.